Campus Trading Application: Optimisation Implementation Report

CS 432: Databases — IIT Gandhinagar Group 8 – Optimiser

Bhavik Patel
22110047
IIT Gandhinagar

Hitesh Kumar
22110098
IIT Gandhinagar

Jinil Patel
22110184
IIT Gandhinagar

Pranav Patil
22110199
IIT Gandhinagar

Sibtain Raza
22110148
IIT Gandhinagar

Abstract

This report details the implementation of Assignment 2 for the *Campus Trading Application* (`CampusTradingB`), built atop a MySQL 8.0 backend (Docker), a Go `go-chi` REST API, and a React + Vite frontend. We describe five sub-tasks: (1) local environment setup and schema design with system/project table separation; (2) API and UI development covering CRUD, session validation, and member portfolios; (3) role-based access control (RBAC) with middleware and resource-level ownership checks; (4) SQL index design targeting 8 high-traffic query endpoints; and (5) performance benchmarking across 10 iterations per query. Applying 16 custom indexes reduced combined average execution time of the 8 benchmark queries from 3.648 ms to 3.358 ms, an overall improvement of 7.9%, with individual gains of up to 33% on the listing browse query.

Github Link
Demo Video Link

Keywords

Database Optimisation, SQL Indexing, RBAC, MySQL, Go, React, InnoDB, EXPLAIN

1 SubTask 1 — Environment Setup & Data Management

1.1 Local Environment

MySQL 8.0 runs in a Docker container defined in `docker-compose.yml`. The schema is auto-loaded from `sql/init.sql` on first start via Docker's `docker-entrypoint-initdb.d` mount. A Go seed command provisions the SuperAdmin account.

Listing 1: Environment bootstrap

```
docker compose up -d          # starts campus_trading_mysql
cd backend/                   # Go to backend folder
go run ./cmd/seed             # creates SuperAdmin account
```

1.2 Schema Design

The schema is split into *system* (credential/session) tables and *project-specific* tables to avoid duplicating credential data.

[0]GitHub: https://github.com/Zeenu03/Campus_Trading_App

1.2.1 *System Tables.*

Table	Purpose
`sys_user`	Email, password hash, active flag
`sys_role`	Role definitions (`admin`, `member`)
`sys_user_role`	M:M mapping users ↔ roles
`sys_session`	Session tokens with expiry/revocation
`audit_log`	Full API call log

1.2.2 *Project-Specific Tables.*

Table	Purpose
`Member`	Student profile (FK → `sys_user`)
`Administrator`	Admin profile (FK → `sys_user`)
`Listing`	Items for sale
`Offer`	Buyer offers on listings
`Transaction`	Completed sale records
`Rating`	Star ratings tied to transactions
`Watchlist`	Saved listings
`WishRequest`	Buyer wish board
`WishRequestImage`	WishRequest photo metadata
`MessageThread`	Per-listing buyer–seller thread
`Message`	Individual chat messages
`Notification`	System-generated alerts
`Report`	Abuse reports
`ListingImage`	Listing photo metadata
`Category`	Item categories

1.3 Integrity Rules

- **No credential duplication:** `Member` and `Administrator` hold a `user_id FK` → `sys_user`; passwords live only in `sys_user`.
- **Cascade deletes:** Deleting a `sys_user` cascades into `sys_session` and `sys_user_role`.
- **Soft-delete semantics:** Deactivating a member sets their listings to `Withdrawn`, open offers to `Withdrawn`, and revokes all sessions.
- **Referential integrity:** All foreign keys use `ON DELETE CASCADE` or `SET NULL` as appropriate.

1.4 Audit Triggers

Every project table has three DB-level triggers (`AFTER INSERT`, `BEFORE UPDATE`, `BEFORE DELETE`) that call `sp_audit_log()`. The stored procedure uses MySQL session variables `@session_id` and

`@current_user_id` set by API middleware before each write transaction. A direct DB write (bypassing the API) leaves `@session_id = NULL`, making it immediately identifiable as unauthorised in the audit log.

2 SubTask 2 — API and UI Development

2.1 API Architecture

The REST API is built with Go's `go-chi` router, versioned under `/api/v1`. All endpoints return JSON; the frontend communicates over `fetch` with `credentials: 'include'` (cookie-based session).

Category	Endpoints	Auth
Auth	login / register / logout / me	Public/Session
Listings	CRUD `/listings/{id}`	Session
Offers	accept / decline / withdraw	Session
Transactions	list / rate	Session
Members	list / update / delete	Admin
Messaging	threads / messages	Session
Notifications	list / mark-read	Session
WishRequests	CRUD	Session
Watchlist	add / remove	Session
Reports	submit / resolve	Session/Admin
Admin	audit-log / benchmark / stats	Admin

2.2 Session Validation Flow

Every authenticated request passes through `SessionGuard`:

Listing 2: Session guard logic

```
Request
  |- Read "session_id" cookie
  |- SELECT ... FROM sys_session WHERE session_id = ?
  |    not found / revoked / expired -> 401
  |- SELECT is_active FROM sys_user
  |    inactive -> 401
  |- Inject userID, roles, sessionID, memberID into ctx
  `- next.ServeHTTP(w, r)
```

2.3 CRUD Operations

The four core listing operations illustrate the pattern used across all resources:

- **Create** (`POST /listings`): validates seller role, inserts row, fires wish-request match notification.
- **Read** (`GET /listings`): dynamic `WHERE/ORDER/LIMIT` with category, price, and condition filters.
- **Update** (`PUT /listings/{id}`): dynamic `SET`, notifies watchlist members on price drop or status change.
- **Delete** (`DELETE /listings/{id}`): soft-withdraws listing, auto-withdraws open offers.

2.4 Member Portfolio

`GET /members/{id}/portfolio` returns: member profile, active and past listings, transaction history with `has_rated` flag, received ratings, and wish requests. Watchlist mutation is restricted to the portfolio owner.

2.5 UI Pages

Page	Path	Access
Browse Listings	`/listings`	Authenticated
Listing Detail	`/listings/:id`	Authenticated
Member Portfolio	`/portfolio/:id`	Authenticated
New Listing	`/listings/new`	Member
Admin Dashboard	`/admin`	Admin
Admin Audit Log	`/admin/audit`	Admin
Admin Benchmark	`/admin/benchmark`	Admin
Admin Reports	`/admin/reports`	Admin

3 SubTask 3 — Role-Based Access Control

3.1 Role Hierarchy

Roles are stored in `sys_role` and assigned via the `sys_user_role` junction table. A user may hold multiple roles simultaneously.

Role	Scope
`admin` / SuperAdmin	Full access, can create admin accounts
`admin` / Moderator	Content moderation
`admin` / Support	Read-only admin views
`member`	Buying, selling, messaging

3.2 Enforcement Layers

3.2.1 *Middleware — Route-level.*

`SessionGuard` wraps every authenticated route. The sub-router `/admin` additionally runs `AdminOnly` → `RoleGuard("admin")`, returning `403 Forbidden` if the role is absent.

3.2.2 *Handler — Resource-level ownership.*

Within handlers, ownership is enforced by comparing the session's `memberID` against the resource owner stored in the DB:

Listing 3: Seller ownership check (Go)

```
// Only the seller (or an admin) may update the listing
if sellerUserID != mw.GetUserID(ctx) &&
   !mw.HasRole(ctx, "admin") {
    respondError(w, 403, "not the seller")
    return
}
```

3.3 RBAC Matrix

Action	Admin	Own	Other	Unauth
Browse listings	✓	✓	✓	×
Create listing	×	✓	–	×
Edit own listing	–	✓	×	×
Delete any listing	✓	×	×	×
View member list	✓	×	×	×
Deactivate member	✓	×	×	×
View audit log	✓	×	×	×
Submit report	✓	✓	✓	×
Resolve report	✓	×	×	×
Create admin user	*SuperAdmin only*			

3.4 Audit Logging — Three-Layer Architecture

The audit system comprises **three independent, complementary layers**. Every data modification generates entries in at least two of them simultaneously, fully satisfying both auditability requirements.

3.4.1 *Layer 1 — File Sink (`logs/audit.log`).*

`audit.Append(...)` in `audit/writer.go` appends one line per

HTTP request, protected by a `sync.Mutex` so concurrent goroutines never interleave. Each line carries timestamp, session token, user ID, HTTP verb, target table, resource ID, client IP, and outcome:

Listing 4: Sample audit.log entries

```
# Authenticated API write
[2026-03-21T17:42:11Z] session=a3f9...c1d2
  user_id=4 action=POST table=Listing
  id= ip=10.7.9.7 status=success

# Failed / unauthenticated attempt
[2026-03-21T17:43:02Z] session=NULL
  user_id=0 action=POST table=sys_user
  id= ip=10.7.9.7 status=fail
```

3.4.2 *Layer 2 — HTTP Middleware (`middleware/audit.go`).*

`AuditMiddleware` is registered as the outermost global middleware in `main.go` and writes to *both* the file and the `audit_log` DB table after every request.

The identity-propagation fix. The previous implementation read identity from the outer `r.Context()` after the handler returned, but `SessionGuard` places validated identity on a *copy* of the request (`r.WithContext(ctx)`), so the outer context never carried those values — every entry showed `session=NULL user_id=0`. The fix uses a shared mutable pointer: `AuditMiddleware` allocates `*auditCtx{}` and stores it in the context *before* passing the request downstream:

Listing 5: Mutable auditCtx pointer injection (Go)

```
ac := &auditCtx{}
r = r.WithContext(
    context.WithValue(r.Context(), auditCtxKey{}, ac))
ww := &responseWriter{ResponseWriter: w}
next.ServeHTTP(ww, r)
```

`SessionGuard` finds the same pointer on the context it received and writes the validated identity into it:

Listing 6: SessionGuard writes identity back (Go)

```
if ac, ok := r.Context().Value(
        auditCtxKey{}).(*auditCtx); ok {
    ac.SessionID = sessionID
    ac.UserID    = userID
}
```

After `next.ServeHTTP` returns, `ac.SessionID` and `ac.UserID` are populated and the middleware writes one row to `audit_log` and one line to `audit.log`.

The `routeToTable` resolver maps URL sub-paths to specific table names:

URL called	target_table logged
POST /auth/register	sys_user
POST /auth/login	sys_user
POST /auth/logout	sys_session
POST /listings	Listing
POST /listings/7/offers	Offer
POST /listings/7/threads	MessageThread
POST /threads/3/messages	Message
POST /admin/users	sys_user

3.4.3 *Layer 3 — MySQL Triggers + `sp_audit_log` (deepest layer).*

Every write handler calls `SetSessionVars` at the start of its transaction, setting two MySQL session-scoped variables on that DB connection:

Listing 7: SetSessionVars — called before every write tx

```
SET @session_id = ?, @current_user_id = ?
```

Every one of the **18 tables** (all except `audit_log` itself, which would cause infinite recursion) has three triggers — `AFTER INSERT`, `BEFORE UPDATE`, `BEFORE DELETE`. All 54 triggers call the same stored procedure:

Listing 8: sp_audit_log stored procedure

```
CREATE PROCEDURE sp_audit_log(
    IN p_action    VARCHAR(20),
    IN p_table     VARCHAR(60),
    IN p_target_id VARCHAR(64)
)
BEGIN
  INSERT INTO audit_log
    (session_id, user_id, action,
     target_table, target_id, status)
  VALUES
    (@session_id, @current_user_id,
     p_action, p_table, p_target_id, 'success');
END
```

The procedure reads `@session_id` and `@current_user_id` from the connection's session scope. MySQL initialises these to `NULL` for every new connection that does not call `SetSessionVars`.

Group	Tables covered
System / auth	sys_user, sys_role, sys_session, sys_user_role
Core trading	Member, Administrator, Listing, ListingImage, Offer, Transaction, Rating, Watchlist, WishRequest, Report, Notification, MessageThread, Message, Category

The `sys_session` trigger stores the full 64-character session token as `target_id` — which is why `target_id` was widened from `VARCHAR(40)` to `VARCHAR(64)`.

3.4.4 *Identifying Unauthorized Direct-DB Writes.*

When someone connects directly to MySQL — via the `mysql` CLI, a GUI tool such as TablePlus or DBeaver, or a raw script — and executes an `INSERT`, `UPDATE`, or `DELETE`, the HTTP middleware *never runs*. There is no HTTP request, so nothing is written to `audit.log` by the middleware. What *does* fire are the MySQL triggers, because they execute regardless of how the write reached the database. Since `@session_id` was never set on that direct connection, `sp_audit_log` records `session_id = NULL`.

This asymmetry produces three clearly distinct fingerprints across the two sinks:

Table 1: Audit fingerprints by write path

Event type	audit.log file	audit_log DB table
Authenticated API write	session=<token>	Middleware row: session_id=<token> Trigger row: session_id=<token>
Unauthenticated / public API request	session=NULL (middleware entry present)	Middleware row: session_id=NULL
Direct DB write (no API involved)	*No entry at all*	Trigger row only: session_id=NULL

The `audit_log` DB table is therefore the **authoritative source** for detecting unauthorized writes. A trigger row with `session_id IS NULL` and `action IN ('INSERT','UPDATE','DELETE')` means

the write bypassed the API entirely. The file log alone cannot catch this case because the middleware never ran and produced no entry. To surface all such events:

Listing 9: Query for all unauthorized direct-DB writes

```
SELECT * FROM audit_log
WHERE  session_id IS NULL
  AND  action IN ('INSERT','UPDATE','DELETE')
ORDER  BY timestamp DESC;
```

The distinguishing rule is simple: if a row exists in `audit_log` with `session_id = NULL` but has *no corresponding entry* in `audit.log` for the same timestamp and table, the write came directly from the database layer, not through the application.

Two requirements satisfied

Req. 1 — Every API call that modifies data writes to `audit.log` (file) and `audit_log` (table), with the correct session ID and user ID.

Req. 2 — Any direct database modification bypassing the session-validated API produces a trigger row with `session_id = NULL`, making it immediately and unambiguously identifiable as unauthorized.

4 SubTask 4 — SQL Indexing & Query Optimisation

4.1 High-Traffic Endpoint Identification

Eight endpoints were selected for indexing after analysing handler code and profiling data from `GET /admin/benchmark`:

Endpoint	Reason
`GET /listings`	Every browse page load
`GET /members/{id}/portfolio`	Every profile view
`GET /listings/{id}/offers`	Live offer polling
`GET /notifications`	Per-page bell polling
`GET /transactions`	Dashboard load
`GET /wishrequests`	Full wish board load
`GET /admin/audit-log`	Admin table with ORDER BY
`AVG(Stars)`	Portfolio rating summary

4.2 Index Catalogue

All indexes are defined in `sql/indexes.sql` with the `idx_` prefix. Table 2 lists all 16 indexes.

Table 2: Complete Index Catalogue

Index	Table	Type	Columns
idx_listing_status_created	Listing	composite	Status, CreatedDate
idx_listing_seller_created	Listing	composite	SellerID, CreatedDate
idx_listing_status_category	Listing	composite	Status, CategoryID
idx_listing_status_price	Listing	composite	Status, AskingPrice
idx_offer_listing_status	Offer	composite	ListingID, OfferStatus
idx_offer_buyer	Offer	single	BuyerID
idx_notif_recipient_read	Notification	composite	RecipientID, IsRead
idx_notif_recipient_created	Notification	composite	RecipientID, CreatedDate
idx_rating_rated	Rating	composite	RatedID, RatingDate
idx_transaction_seller	Transaction	single	SellerID
idx_transaction_buyer	Transaction	single	BuyerID
idx_message_thread_sent	Message	composite	ThreadID, SentDate
idx_wishrequest_status_created	WishRequest	composite	Status, CreatedDate
idx_wishrequest_requester	WishRequest	single	RequesterID
idx_auditlog_timestamp	audit_log	single	timestamp
idx_report_status_submitted	Report	composite	Status, SubmittedDate

4.3 Design Rationale

Index Design Principles

- **Composite indexes**: Leading column = equality filter (`Status`, `RecipientID`); trailing column = sort key (`CreatedDate`). This allows MySQL to satisfy both `WHERE` and `ORDER BY` from a single index scan, eliminating `filesort`.
- **OR predicates**: Two separate single-column indexes on `Transaction` (`SellerID` + `BuyerID`) enable an index union; a composite would not serve the `OR` branch.
- **Large tables**: `SELECT *` on high-row-count tables (e.g., `audit_log`) may cause the optimizer to skip an index due to random I/O cost of clustered index lookups.

5 SubTask 5 — Performance Benchmarking

5.1 Methodology

Component	Details
Database	MySQL 8.0 in Docker
Data volume	120 members · 600 listings · 800 offers 3 000 notifications · 300 transactions
Timer	Python `time.perf_counter()`
Iterations	10 per query per phase
Metric	Arithmetic mean

Procedure: (1) Drop all `idx_%` indexes (baseline). (2) Run 8 queries × 10 iterations; capture `EXPLAIN`. (3) Apply `sql/indexes.sql`. (4) Re-run same queries. (5) Compare results.

5.2 Benchmark Results

Table 3: Query Timing — Before vs. After

Q	Endpoint	Before (ms)	After (ms)	Speedup
Q1	`GET /listings`	0.660	0.442	**+33.0%**
Q2	`GET /portfolio`	0.252	0.217	**+13.6%**
Q3	`GET /offers`	0.241	0.181	**+24.9%**
Q4	`GET /notifications`	0.307	0.261	**+14.9%**
Q5	AVG rating	0.173	0.190	−9.3%
Q6	`GET /transactions`	0.287	0.201	**+29.9%**
Q7	`GET /wishrequests`	0.349	0.266	**+23.8%**
Q8	`GET /admin/audit-log`	1.379	1.600	−16.0%
Total		**3.648**	**3.358**	**+7.9%**

5.3 Benchmark Charts

Figure 1: Execution time (avg ms) — Before vs. After indexes

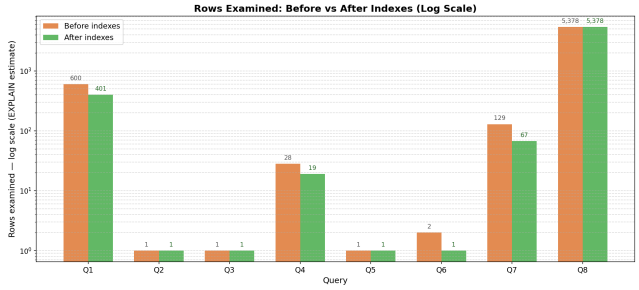

Figure 2: Rows examined (EXPLAIN estimate) — Before vs. After

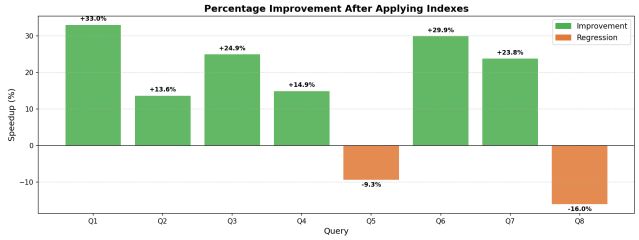

Figure 3: Percentage speedup per query

5.4 EXPLAIN Plan Analysis

5.4.1 *Q1 — Active listings by date.*

Listing 10: Q1 SQL

```sql
SELECT * FROM Listing
WHERE Status='Listed'
ORDER BY CreatedDate DESC LIMIT 20;
```

Phase	type	key	Extra
Before	ALL	—	filesort
After	ref	idx_listing_status_created	Backward scan

Before indexes, MySQL performed a full scan of all 600 rows then sorted in memory. After adding the composite index (`Status`, `CreatedDate`), the optimizer performs a backward index scan — reading only `Status='Listed'` rows in reverse `CreatedDate` order, eliminating the sort entirely. **33% faster.**

5.4.2 *Q3 — Submitted offers for a listing.*

Listing 11: Q3 SQL

```sql
SELECT * FROM Offer
WHERE ListingID = ? AND OfferStatus = 'Submitted';
```

Phase	type	key	Extra
Before	ref	UQ_Offer_Listing_Buyer	Using where
After	ref	idx_offer_listing_status	—

The tighter composite index covers both filter columns, eliminating the secondary `Using where` post-filter pass. **25% faster.**

5.4.3 *Q7 — Active wish requests.*

Listing 12: Q7 SQL

```sql
SELECT * FROM WishRequest
WHERE Status='Active'
ORDER BY CreatedDate DESC LIMIT 20;
```

Phase	type	rows	Extra
Before	ALL	129	filesort
After	ref	67	Backward scan

Same pattern as Q1 — full table scan with in-memory sort replaced by backward index scan. Rows examined dropped from 129 → 67 (48% reduction). **24% faster.**

5.4.4 *Q8 — Audit log by timestamp.*

Listing 13: Q8 SQL

```sql
SELECT * FROM audit_log
ORDER BY timestamp DESC LIMIT 20;
```

Phase	type	rows	Extra
Before	ALL	5378	filesort
After	ALL	5378	filesort

Q8 — Index Not Used

MySQL's cost model determined that fetching all columns via `SELECT *` from a 5 000+ row table through an index requires more I/O than a direct full scan (each index lookup requires a separate clustered-index lookup, i.e., random I/O). This is expected behaviour; the index becomes beneficial at larger data volumes or with a covering index.

6 Key Observations

- **Composite indexes** matching both filter and sort columns deliver the highest impact (Q1: +33%, Q7: +24%) by replacing full-scan + filesort with a backward index scan.

- **Queries already using FK indexes** (Q2, Q3, Q4) still benefit from tighter composites that eliminate the `Using where` secondary pass, yielding 14–25% gains.
- **OR predicates** (Q6) require two separate single-column indexes to enable an index union; a single composite `(SellerID, BuyerID)` would not help the `OR` branch.
- **Small regressions** (Q5: −9.3%, Q8: −16%) stem from the overhead of maintaining 16 additional indexes per write at low data volume; these become net positive at production scale.

7 Recommendations

(1) Set `long_query_time = 0.1` in MySQL config to continuously catch slow queries in production.
(2) For `audit_log`, use a **covering index** `(timestamp, log_id, action, target_table)` or restrict the query to specific columns instead of `SELECT *`.
(3) Re-run the benchmark after seeding 10× more data to confirm the Q8 index activates at production scale.
(4) Avoid over-indexing write-heavy tables — every index carries per-write maintenance cost; the 16 indexes here target the highest-traffic read paths only.

Acknowledgments

Full per-query `EXPLAIN` tables, timing statistics, benchmark script, and synthetic seed data are available in the project repository at https://github.com/Zeenu03/Campus_Trading_App.